

Aravind Pradeep

AI Engineer | Agentic Systems & RAG Pipelines | LLMOps & Evaluation
Cottbus, Germany (open to relocate) | +49 176 67314504 | aravindpradeep001@gmail.com
linkedin.com/in/aravind-pradeepmadathinal | github.com/axon011 | aravindpradee.me
English (C1) | German (B1)

PROFESSIONAL SUMMARY

AI Engineer with two years at Perinet building the backend of an industrial-IoT platform: Python and Go services over MQTT streams, FastAPI, Docker, Kubernetes, and CI/CD, plus the model-benchmarking and marketing-analytics workstreams. First author of arXiv:2607.02612 on adaptive inference for Vision Transformers: staged token merging and pruning that cuts inference energy 48% on ImageNet-1k. Independently I design, ship, and maintain RAG, agent, and LLMops systems: hybrid retrieval (BM25 + dense with reciprocal rank fusion) with RAGAs and MLflow evaluation, LangGraph and GraphRAG agent pipelines, QLoRA fine-tuning, and call-level observability. Nearly three years of professional software experience in total, including earlier enterprise work at Cognizant. Finishing my M.Sc. in AI at BTU Cottbus (thesis on Vision Transformer optimization for edge devices).

PROFESSIONAL EXPERIENCE

AI Engineer

Perinet GmbH | Cottbus, Germany | Jul 2024 – May 2026 (23 months)

- Developed Python and Go backend services connecting LLM workflows to live MQTT sensor streams, exposed as versioned FastAPI REST APIs with async endpoints; containerized with Docker/Kubernetes and automated CI/CD via GitHub Actions for zero-touch production releases.
- Led the containerization workstream for the AI Container umbrella project — PeriChat (Go-based chatbot with MapReduce retrieval), MessageSense (streaming MQTT message anomaly detection), and PeriXplore (sensor data exploration); sole maintainer of several of these services end to end, handling profiling, QA, and production deployment integration with the engine and ML teams.
- Owned the RAG evaluation and model-benchmarking workstream; built the hybrid retrieval and evaluation layer behind PeriChat and measured retrieval speed, generation quality, and trade-offs across model variants for the AI container platform, informing downstream chatbot and corpus design.
- Built a marketing-analytics pipeline (CEO-sponsored) pulling LinkedIn and Google Analytics 4 data through MQTT into a dashboard, with an event-matching script that links campaigns to engagement; cut reporting lag from roughly 48 hours to a daily refresh.

Software Engineer Trainee

Cognizant Technology Solutions | India | Oct 2021 – May 2022

- Developed and maintained mainframe applications (COBOL, JCL, DB2) in agile sprints; built structured debugging practices and production deployment discipline across enterprise banking systems.

PROJECTS

Wind-Farm Installation Planner | Python, NumPy, pandas, Pydantic, FastAPI

- Built a weather-aware scheduler for a 12-turbine wind-farm build that searches 48 candidate campaigns against 20 years of hourly wind records, pricing each as a risk distribution rather than a single point estimate.
- Cut the plan from EUR 5.59M over 189 days to EUR 3.56M over 64 days against the naive baseline, feasible in 19 of 19 held-out weather years; the scheduling maths is fully deterministic, with the LLM confined to the human edges.

Multi-Agent Research & Report Pipeline | LangGraph, CrewAI, FastAPI, Docker, GitHub Actions

- Architected a 4-agent pipeline (Planner, Researcher, Writer, Critic) using LangGraph state machines with strict role boundaries and Pydantic structured outputs, producing 2,000+ word research reports with verifiable source citations.
- Deployed via async FastAPI with Docker containerization and GitHub Actions CI/CD, automating the path from code push to deployment with no manual steps.

RAG Evaluation System | FastAPI, Qdrant, LangChain, RAGAs, MLflow, BM25

- Engineered a hybrid retrieval pipeline (dense embeddings + BM25 + Reciprocal Rank Fusion) over a 50-topic corpus using Qdrant, reaching 0.94 hit@5 and 0.96 citation presence on a 50-question evaluation set, with async embedding pre-computation and semantic caching.
- Built an automated evaluation pipeline measuring faithfulness, relevance, and context recall via RAGAs; tracked prompt and retrieval experiments in MLflow with regression alerts when retrieval quality dropped below baseline.

GraphRAG Agent | Python, networkx, LLM Entity/Relation Extraction, Docker

- Built a GraphRAG pipeline that constructs a typed, deduplicated knowledge graph from a document corpus via LLM entity/relation extraction (networkx), so retrieval follows explicit relationships instead of chunk-embedding similarity alone.
- Implemented k-hop subgraph retrieval producing grounded answers cited back to source, handling multi-document and relationship-spanning questions that flat RAG misses; LLM backend pluggable across Claude, Codex, and Gemini with a test suite.

GraphRAG Studio | Next.js, TypeScript, React, FastAPI, Python, GraphRAG

- Built a full-stack GraphRAG application: upload documents, watch a typed knowledge graph build as they are processed, then chat over it with k-hop subgraph retrieval and answers cited back to source.
- Shipped a Next.js (App Router) and TypeScript front end with an interactive force-graph visualization over a FastAPI backend that wraps the graphrag-agent package.

Deutsch-Tutor — Adaptive AI German Tutor | tutor.aravindpradee.me | Next.js, TypeScript, Tailwind, SSE Streaming, Zod, LLM Provider Seam

- Built a full-stack AI German tutor (Next.js App Router, TypeScript) that logs every grammar mistake from live conversation and generates personalized drills from them; five tabs span 32 CEFR-graded lessons (A1–B2), streaming chat, practice, a progress recommender, and a 40-rule grammar reference.
- Designed a two-agent-per-turn flow — the reply streams instantly over SSE while a parallel Corrector returns Zod-validated exact-substring error spans — behind a provider seam that swaps the LLM backend (Claude CLI locally, Gemini when deployed) via one env var; “what to practice” is ranked by an explainable recency-weighted formula, not an LLM.

LLM Fine-Tuning: Structured JD Extractor | QLoRA, Qwen2-0.5B, PyTorch, PEFT, 4-bit Quantization

- Fine-tuned Qwen2-0.5B with QLoRA (4-bit NF4, LoRA rank 16 / alpha 32 on attention projections) to extract structured JSON from job descriptions; reached 100% JSON validity and 70%+ entity-field accuracy on a held-out test set, trained in under 4 minutes on a 4GB GPU.
- Trained 2.16M adapter parameters (0.44% of 496M total) and evaluated per-field accuracy across entity and list fields, exposing where list-field F1 needed more training data.

LLMOps Observability Dashboard | FastAPI, React, TypeScript, PostgreSQL, Docker, GitHub Actions

- Built a full-stack LLM observability dashboard that traces every API call to PostgreSQL with latency, token counts, and per-model cost (GPT-4o, Claude, Gemini, DeepSeek) — self-hosted, no external tracing service — plus a session explorer with prompt/response previews.
- Shipped with a multi-stage Docker build, GitHub Actions CI, and 12 tests over pricing, data parsing, and API routes; a React + Recharts front end reads token analytics and cost breakdowns per session.

Resume Tailor — LLM Pipeline with Guardrails | Python, LLM CLIs (Claude/Codex/Gemini), Regex/AST Gates, LaTeX/PDF

- Built a Python CLI that tailors a résumé and cover letter to a target job description with an LLM, then gates the output through automated checks before it compiles: a regression gate that blocks banned or fabricated claims, and a claim-backing gate that traces every skill and metric back to a source résumé.
- Added an ATS scorer (keyword and structural coverage plus an interview-probability estimate) and a humanizer pass that flags AI-writing tells; the LLM backend is pluggable across Claude, Codex, and Gemini CLIs with fallback, rendering to LaTeX/PDF with a text-layer verification step.

TECHNICAL SKILLS

AI & Agents: LangGraph, LangChain, CrewAI, RAG, GraphRAG, Prompt Engineering, Structured Outputs (Pydantic), Tool Use, Function Calling, MCP

LLMOps & Evaluation: Langfuse (Tracing, Evals), RAGAs (Faithfulness, Relevance, Recall), MLflow, LLM-as-Judge, A/B Prompt Evaluation, Regression Detection, Guardrails

ML & NLP: Python, PyTorch, Hugging Face Transformers, scikit-learn, Fine-Tuning (QLoRA, PEFT), Model Compression (Quantization, Pruning), Edge AI Optimization, Chunking, Embeddings, NER, Computer Vision, Explainable ML

LLMs & APIs: OpenAI, Anthropic (Claude), Gemini, DeepSeek, Open-Source LLMs (Qwen, DeepSeek, Llama)

Backend & Infra: Go, FastAPI, REST/gRPC, API Integrations, Docker, Kubernetes, Linux, Vector Databases, GitHub Actions, GitLab CI/CD, PostgreSQL, Qdrant, ChromaDB, OpenSearch, Elasticsearch, Azure, AWS (S3, Bedrock, EC2)

Frontend: React, TypeScript, Next.js, Recharts

Languages: English (C1 Fluent), German (B1 Actively Improving), Malayalam (Native)

PUBLICATIONS

- **Fusion: A Framework for Unified Sequential Token Adaptation in Vision Transformers.** A. Pradeep, S. Nazari, M. Taheri, C. Herglotz. Accepted at DSD 2026. [arXiv:2607.02612](https://arxiv.org/abs/2607.02612) [cs.CV], Jul 2026.
- Stages token merging, early exiting, and token pruning so they cooperate rather than destabilize each other; routing modules adapt compression per input and shift the accuracy/latency trade-off at inference time without retraining. On ImageNet-1k with DeiT-S it matches or surpasses state-of-the-art adaptive ViT methods at comparable compute, cutting calibration error up to 4× and inference energy 48%.

EDUCATION

M.Sc. Artificial Intelligence (Research Profile)

Brandenburg University of Technology | Cottbus, Germany | Oct 2022 – present (thesis phase)

- Thesis: Content-Aware ViT Optimization on Edge Devices. Focus: ML, Computer Vision, Explainable ML, Data Mining.

B.Sc. Computer Application

BVM Holy Cross College | Kottayam, India | Jul 2018 – Mar 2021